

Lab Assignment 2

CMPE 146 Section 1

Professor Bhawandeep Singh Harsh

Andrew Bertch

Robert Trinh

Table of Contents

Exercise 1 – Control Methods	3
Exercise 1.1 – DriverLib	4
Exercise 1.2 – Direct Port Access	6
Exercise 1.3 – Bit-band Access.....	7
Exercise 2 – Control two LEDs	8
Exercise 3 – Control LED with push button.....	10
Appendix A – MCU Source Code.....	12
driverLib_LED.c	12
portAccess_LED.c	14
bitBand_LED.c.....	16
two_LED.c.....	18
button_LED.c.....	20

Exercise 1 – Control Methods

In this exercise we blink an LED on the LaunchPad using various control methods available on the LaunchPad. We will learn how to use the DriverLib functions, direct port, and bit band access. This part of the exercise focuses on the initial set up of the MCU to control the LED and other peripheral devices. We learned to make use of the data sheets of the MCU and the LaunchPad to find devices and their associated pin numbers.

Exercise 1.1 – DriverLib

In this exercise we controlled the LED using the DriverLib functions. We started by using the initial set up from exercise 1. We also added some instructions to set the LED states to known values. This was done outside of the control loop.

```
DL_GPIO_enablePower(GPIOB);
```

```
DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
```

```
DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in  
LED2
```

```
DL_GPIO_initDigitalOutput(IOMUX_PINCM50); // PB22, blue LED in  
LED2
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM50); // PB22, blue LED
```

```
DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
```

```
DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
```

```
DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_22); // PB22, blue LED
```

This section of code is used for setup of the LEDs for use.

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);
```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);
```

This section of code is used for state initialization of the LEDs.

```
while (1) {
```

```
    DL_GPIO_setPins(GPIOB, DL_GPIO_PIN_22);
```

```
    delay_cycles(32000000);  
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);  
    delay_cycles(32000000);  
}
```

This section of code is used for control of the LED.

The control loop works by setting the pin of the blue LED to a value of 1, then waiting for 32,000,000 cycles (1 second), then clearing the pins (pin = 0), then waiting for 32,000,000 cycles (1 second) again, and then looping.

If the value of the delay is too small, it would likely result in the LED always being off. This is because we start with the LED off and the time that a delay of 1,000 would result in would be ~0.00003125 seconds. This would likely be too short of a delay for the LED to physically change states in the hardware. This would result in the hardware starting to change to turn the LED on but the delay ending and the next instruction telling the LED to turn off and thus keeping the LED off.

Exercise 1.2 – Direct Port Access

In this exercise we controlled devices using the memory address of the GPIO register and the bit associated with the pin the LED was on. This required us to look up the GPIO1 base memory address and the offset address for the digital output register. This base address was at 0x400A2000 with the DOUT31_o register offset being 1280h and the DIN31_o register offset being 1380h.

```
uint32_t *DOUT31_0 = (uint32_t*)(0x00001280 + 0x400A2000);
```

```
uint32_t *DIN31_0 = (uint32_t*)(0x00001380 + 0x400A2000);
```

These lines were used to set up the pointers to read and set the register controlling the LED.

```
while (1) {  
    *DOUT31_0 = *DIN31_0 ^ BIT(22);  
    delay_cycles(32000000);  
}
```

The control loop works by setting the output register to the value of input register xor bit 21. That means that bit 21, the bit that controls the LED on pin PB22, would get inverted after each delay since $1 \wedge 1 = 0$ and $0 \wedge 1 = 1$.

```
#define BIT(n) 1 <<(n)
```

This macro was used to make access to the correct bit easier.

Exercise 1.3 – Bit-band Access

In this exercise we make use of bit bands support in the MCU to control the LED. This is done by accessing the byte in the 32-bit register for the 4 bytes that contain pin 22. That means that we want to access the 3rd byte in the DOUT23_20 register. That corresponds to an address equal to 0x400A3216. This was determined by adding the base address for GPIO1, 0x400A2000, with the offset for DOUT23_20, 0x1214, with the third byte for pin 22 being 0x1216. This results in the final alias address of 0x400a3216.

CORTEX_MoP: alias address: 0x400a3216

```
uint8_t *BLED = &((uint8_t*)0x400A3214)[2];
int state = 0;
printf("alias address: 0x%p\n", BLED);

while (1) {
    *BLED = state = !state;
    delay_cycles(32000000);
}
```

The control loop works by setting the pointer for the blue LED equal to the variable state. The value of state is also assigned to the inverse of its current value each time. The delay has the LED blink once a second.

The advantages of controlling the LED in this manner is that we do not need a macro or to count out bits to control the correct device. We have a pointer that we can set the value of to control only that device. This eliminates the possibility of setting the bits for the wrong device.

Exercise 2 – Control two LEDs

In this exercise we control two different LEDs to blink in sync with each other. We control the blue LED in LED2 and LED1. We continue to use the bit-band access method from exercise 1.3 to control the devices. In this exercise, we had to power GPIO0 since LED1 is on pin PA0. This required getting the offset address of GPIO0, 0x400A0000, and DOUT3_0, 0x1200. Below will be the alias addresses for the LEDs. The first one corresponds to the blue LED on LED2 and the second is LED1.

CORTEX_MoP: alias address: 0x400a3216

CORTEX_MoP: alias address: 0x400a1200

```
//invert digital output for LED1

DL_GPIO_initDigitalOutputFeatures(IOMUX_PINCM1,
DL_GPIO_INVERSION_ENABLE, DL_GPIO_RESISTOR_NONE,
DL_GPIO_DRIVE_STRENGTH_LOW, DL_GPIO_HIZ_DISABLE);

uint8_t *BLED = &((uint8_t*)0x400A3214)[2];
uint8_t *LED1 = &((uint8_t*)0x400A1200)[0];

int state = 0;

printf("alias address: 0x%p\n", BLED);
printf("alias address: 0x%p\n", LED1);

while (1) {
    *BLED = state = !state;
    *LED1 = state;
    delay_cycles(32000000);
}
```

The control loop uses nearly the same control loop from exercise 1.3 with the addition of 1 line that sets the pointer for LED1 equal to state.

Both LEDs are made to blink in sync by the addition of the another DriverLib function that inverts the output of the device on pin 0. By using the function

`DL_GPIO_initDigitalOutputFeatures` we can set the digital output to be inverted.

We have to do this since the hardware for LED1 is configured so that 0 turns the LED on and 1 turns it off. By inverting the output, LED1 will be synced with the blue LED on LED2.

[Link to video](#)

Exercise 3 – Control LED with push button

In this exercise we controlled a LED with one of the push buttons on the LaunchPad. Using the bit band access from exercise 1.3, required a byte sized pointer for the button so that we could read the value of the button.

```
DL_GPIO_enablePower(GPIOB);
```

```
DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
```

```
DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in  
LED2
```

```
DL_GPIO_initDigitalOutput(IOMUX_PINCM50); // PB22, blue LED in  
LED2
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM50); // PB22, blue LED
```

```
DL_GPIO_initDigitalInput(IOMUX_PINCM49); // PB21, S2
```

```
DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
```

```
DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
```

```
DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_22); // PB22, blue LED
```

```
DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM49,  
IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,  
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,  
DL_GPIO_WAKEUP_DISABLE);
```

```
uint8_t *BLED = &((uint8_t*)0x400A3214)[2];
```

```
uint8_t *S2 = &((uint8_t*)0x400A3314)[1];
```

These lines of code are for the set up of the LaunchPad.

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);  
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);  
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);  
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_21);
```

```
int state = 0;  
int released = 1;
```

These lines of code are for state initialization of the LaunchPad.

```
while (1) {  
    if (*S2 == 0 && released == 1) {  
        released = 0;  
        state = !state;  
    }  
    else if (*S2 == 1 && released == 0) {  
        released = 1;  
    }  
    *BLED = state;  
}
```

These lines are the control loop for the LED and push button.

The push button S2 we used on the microcontroller will consume more power when pushed in due to the pull up resistor configuration set to read the button. When the button is not pushed in, it sits at a logical high with very little current flowing to the digital input register. When the button is pushed, current starts to flow to ground as the digital value moves to low. With current flowing through the resistor to ground, power is being used, more than was being used with the button not pushed.

[Link to video](#)

Appendix A – MCU Source Code

driverLib_LED.c

```
#include "ti_msp_dl_config.h"

#include <stdio.h>

int main(void)
{
    SYSCFG_DL_init();

    DL_GPIO_enablePower(GPIOB);

    DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
    DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in
LED2
    DL_GPIO_initDigitalOutput(IOMUX_PINCM50); // PB22, blue LED in
LED2

    DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
    DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED
    DL_GPIO_initDigitalInput(IOMUX_PINCM50); // PB22, blue LED

    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_22); // PB22, blue LED

    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);

while (1) {
    DL_GPIO_setPins(GPIOB, DL_GPIO_PIN_22);
    delay_cycles(32000000);
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);
    delay_cycles(32000000);
}
}
```

portAccess_LED.c

```
#include "ti_msp_dl_config.h"

#include <stdio.h>

#define BIT(n) 1 <<(n)

int main(void)
{
    SYSCFG_DL_init();

    DL_GPIO_enablePower(GPIOB);

    DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
    DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in
LED2
    DL_GPIO_initDigitalOutput(IOMUX_PINCM50); // PB22, blue LED in
LED2

    DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
    DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED
    DL_GPIO_initDigitalInput(IOMUX_PINCM50); // PB22, blue LED

    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_22); // PB22, blue LED

    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);
```

```
uint32_t *DOUT31_0 = (uint32_t*)(0x00001280 + 0x400A2000);
```

```
uint32_t *DIN31_0 = (uint32_t*)(0x00001380 + 0x400A2000);
```

```
while (1) {
```

```
    *DOUT31_0 = *DIN31_0 ^ BIT(22);
```

```
    delay_cycles(32000000);
```

```
}
```

```
}
```

bitBand_LED.c

```
#include "ti_msp_dl_config.h"
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    SYSCFG_DL_init();
```

```
    DL_GPIO_enablePower(GPIOB);
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in  
LED2
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM50); // PB22, blue LED in  
LED2
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM50); // PB22, blue LED
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_22); // PB22, blue LED
```

```
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);
```

```
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
```

```
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);
```

```
uint8_t *BLED = &((uint8_t*)0x400A3214)[2];  
int state = 0;  
printf("alias address: 0x%p\n", BLED);  
  
while (1) {  
    *BLED = state = !state;  
    delay_cycles(32000000);  
}  
}
```

two_LED.c

```
#include "ti_msp_dl_config.h"
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    SYSCFG_DL_init();
```

```
    DL_GPIO_enablePower(GPIOB);
```

```
    DL_GPIO_enablePower(GPIOA);
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in  
LED2
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM50); // PB22, blue LED in  
LED2
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM1); // PA0, LED1
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM50); // PB22, blue LED
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM1); // PA0, LED1
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_22); // PB22, blue LED
```

```
    DL_GPIO_enableOutput(GPIOA, DL_GPIO_PIN_0); //PA0, LED1
```

```

DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);
DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_0);

//invert digital output for LED1

DL_GPIO_initDigitalOutputFeatures(IOMUX_PINCM1,
DL_GPIO_INVERSION_ENABLE, DL_GPIO_RESISTOR_NONE,
DL_GPIO_DRIVE_STRENGTH_LOW, DL_GPIO_HIZ_DISABLE);

uint8_t *BLED = &((uint8_t*)0x400A3214)[2];
uint8_t *LED1 = &((uint8_t*)0x400A1200)[0];
int state = 0;
printf("alias address: 0x%p\n", BLED);
printf("alias address: 0x%p\n", LED1);

while (1) {
    *BLED = state = !state;
    *LED1 = state;
    delay_cycles(32000000);
}
}

```

button_LED.c

```
#include "ti_msp_dl_config.h"
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    SYSCFG_DL_init();
```

```
    DL_GPIO_enablePower(GPIOB);
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in  
LED2
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM50); // PB22, blue LED in  
LED2
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM50); // PB22, blue LED
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM49); // PB21, S2
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_22); // PB22, blue LED
```

```
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);
```

```
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_22);
```

```
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_21);
```

```
DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM49,  
IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,  
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,  
DL_GPIO_WAKEUP_DISABLE);
```

```
uint8_t *BLED = &((uint8_t*)0x400A3214)[2];
```

```
uint8_t *S2 = &((uint8_t*)0x400A3314)[1];
```

```
int state = 0;
```

```
int released = 1;
```

```
while (1) {
```

```
    if (*S2 == 0 && released == 1) {
```

```
        released = 0;
```

```
        state = !state;
```

```
    }
```

```
    else if (*S2 == 1 && released == 0) {
```

```
        released = 1;
```

```
    }
```

```
    *BLED = state;
```

```
}
```

```
}
```